DESIGN AND ANALYSIS OF ALGORITHMS

Unit 3: B Trees

Dr. Divyashree N
Department of Computer Science & Engineering

PES UNIVERSITY
CELEBRATING 50 YEARS

DESIGN AND ANALYSIS OF ALGORITHMS

B Trees

B Tree

- Self-balancing search tree
- Each node can contain more than one key
- Can have more than two children.
- All data records (or record keys) are stored at the leaves, in increasing order of the keys each parental node contains $m - 1$ ordered keys.

It is also known as a height-balanced m-way tree.

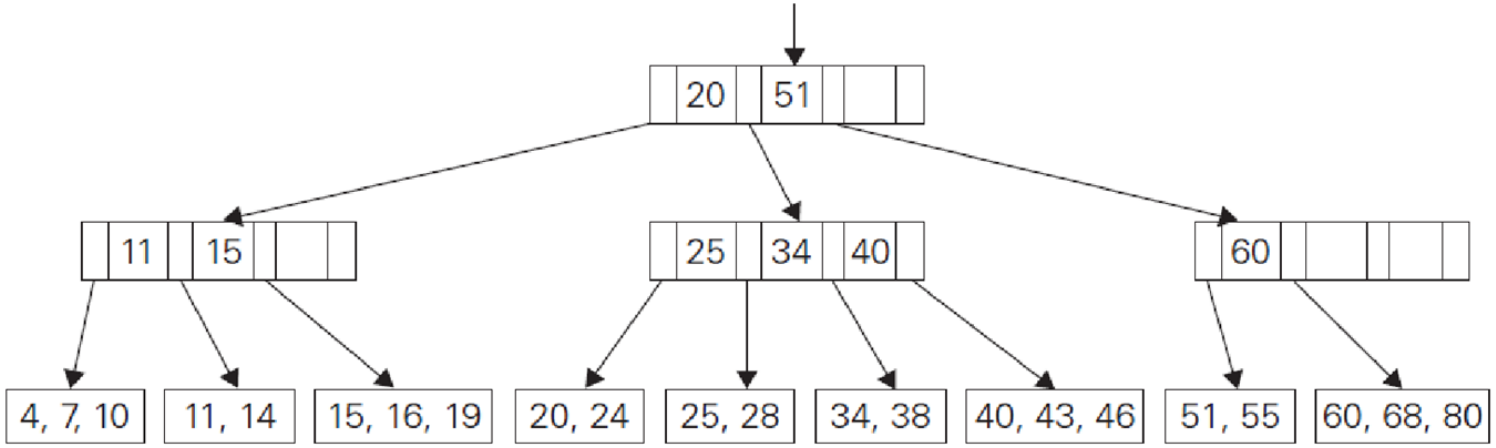


FIGURE 7.8 Example of a B-tree of order 4.

B Tree

Structural properties:

- All leaves are at the same level.
- For each node x, the keys are stored in increasing order.
- If n is the order of the tree, each internal node can contain at most m - 1 keys along with a pointer to each child.
- Each node except root can have at most n children and at least m/2 children.
- All leaves have the same depth (i.e. height-h of the tree).
- The root has at least 2 children and contains a minimum of 1 key.
- If $m \geq 1$, then for any n-key B-tree of height h and minimum degree $t \geq 2$, $h \geq \log_t (m+1)/2$
- B-Tree grows and shrinks from the root which is unlike Binary Search Tree. Binary Search Trees grow downward and also shrink from downward.
- Like other balanced Binary Search Trees, time complexity to search, insert and delete is O(log n).

B Tree

Operations on a B Tree:

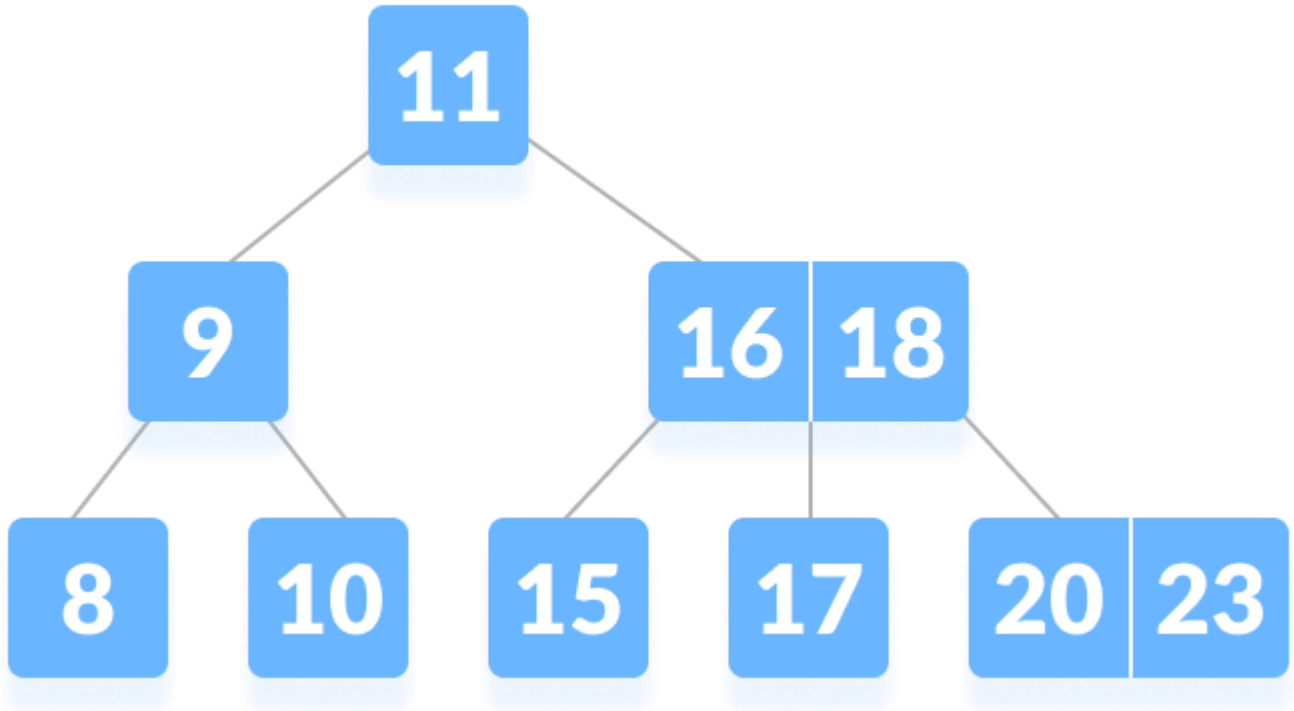
- Searching
- Insertion
- Deletion

Search

1. Starting from the root node, compare k with the first key of the node.
2. If k = the first key of the node, return the node and the index
3. If k.leaf = true, return NULL, i.e. not found
4. If k < the first key of the root node, search the left child of this key recursively.
5. If there is more than one key in the current node and k > the first key compare k with the next key in the node.
6. If k < next key search the left child of this key (ie. k lies in between the first and the second keys). Else, search the right child of the key
7. Repeat steps 1 to 4 until the leaf is reached.

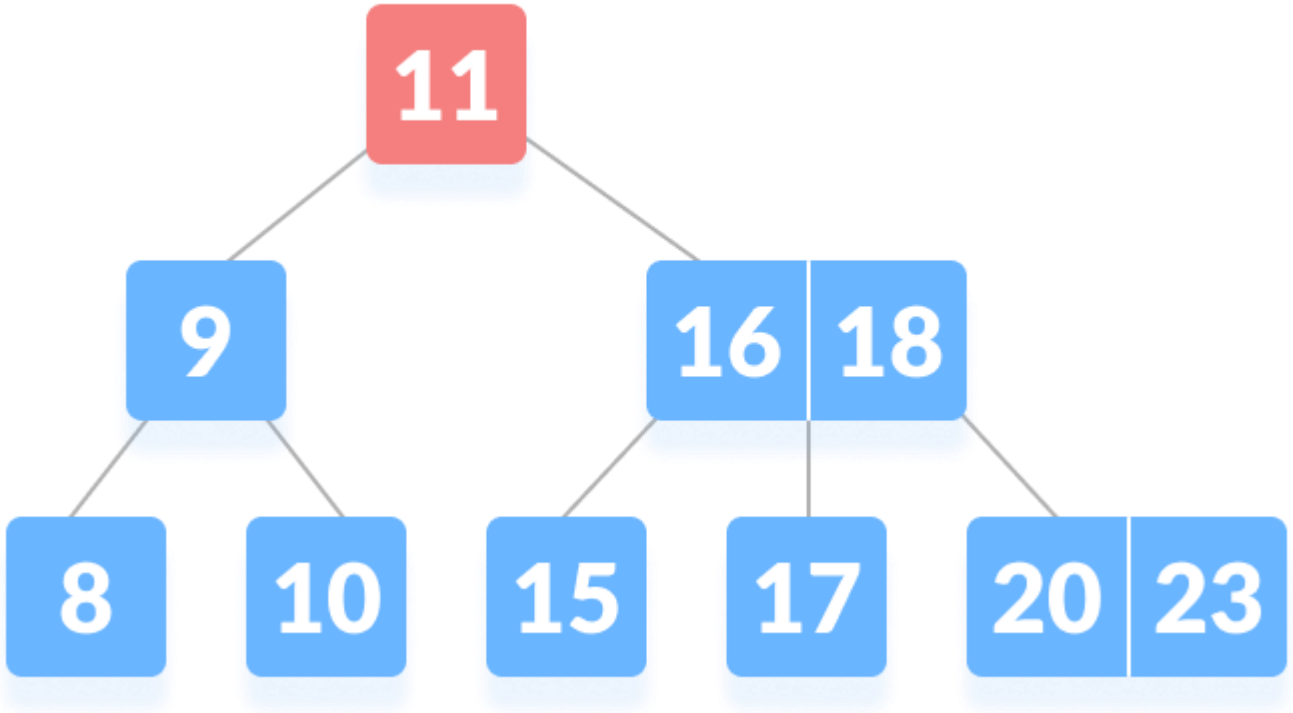
B Tree

Let us search key , k = 17



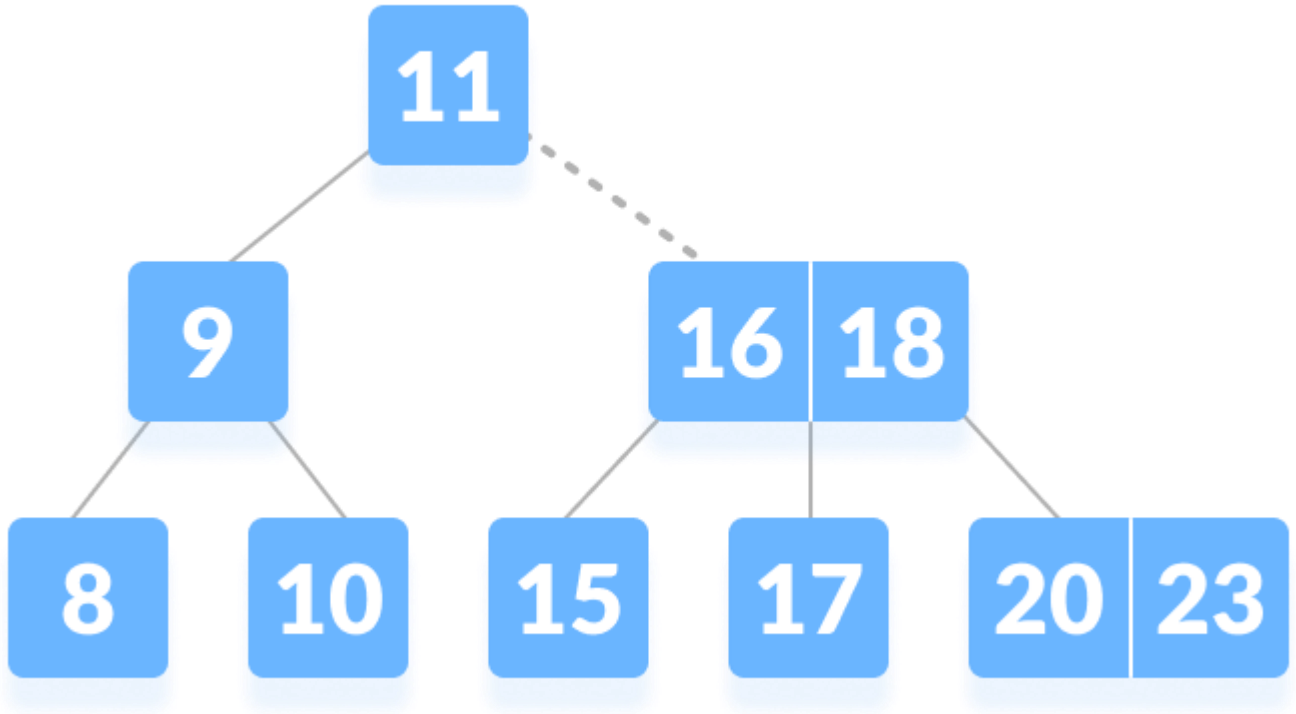
B Tree

17 is not found in the root so, compare it with the root key



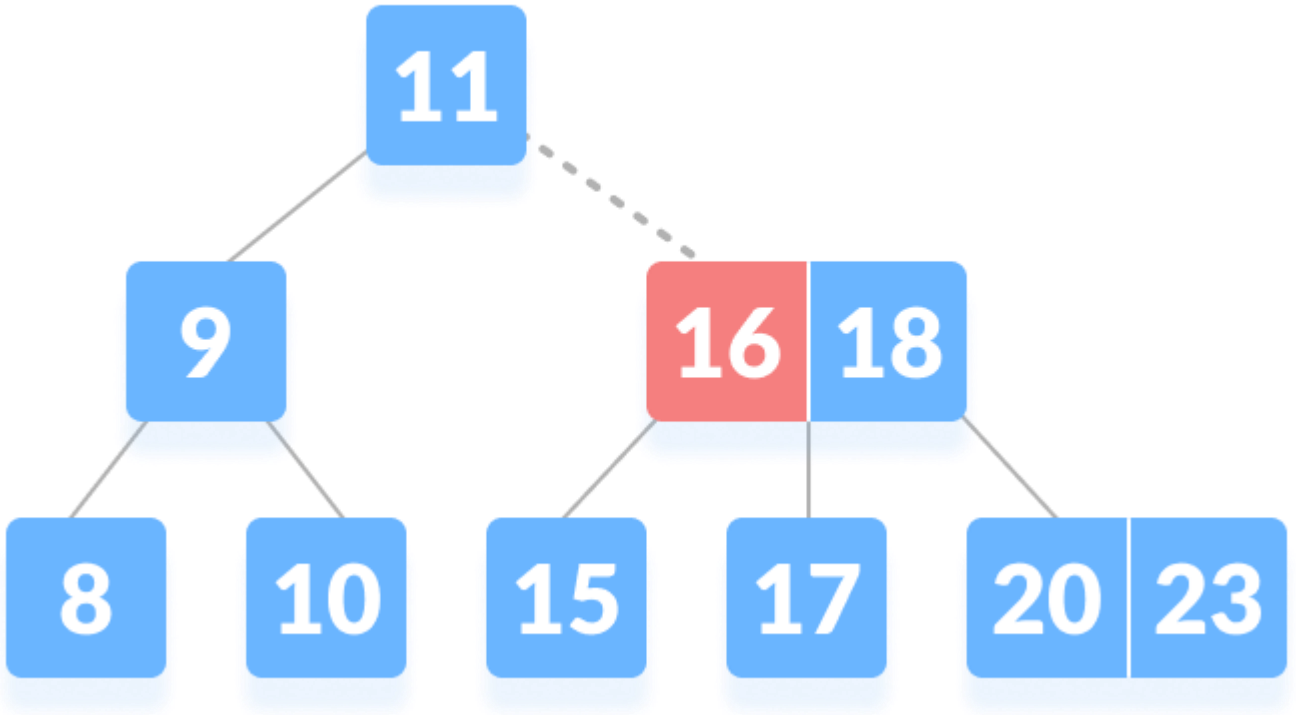
B Tree

Since k > 11 go to the right child of the root node.



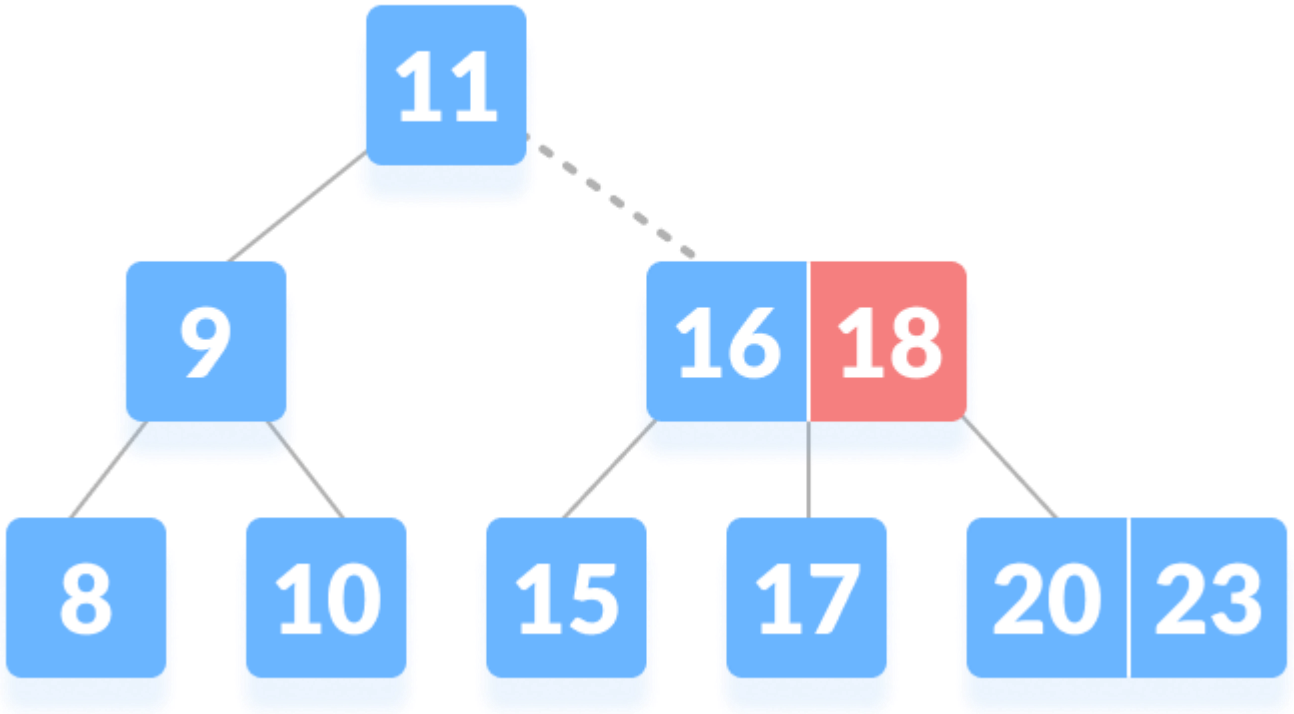
B Tree

Compare k with 16. Since k > 16 compare k with the next key 18



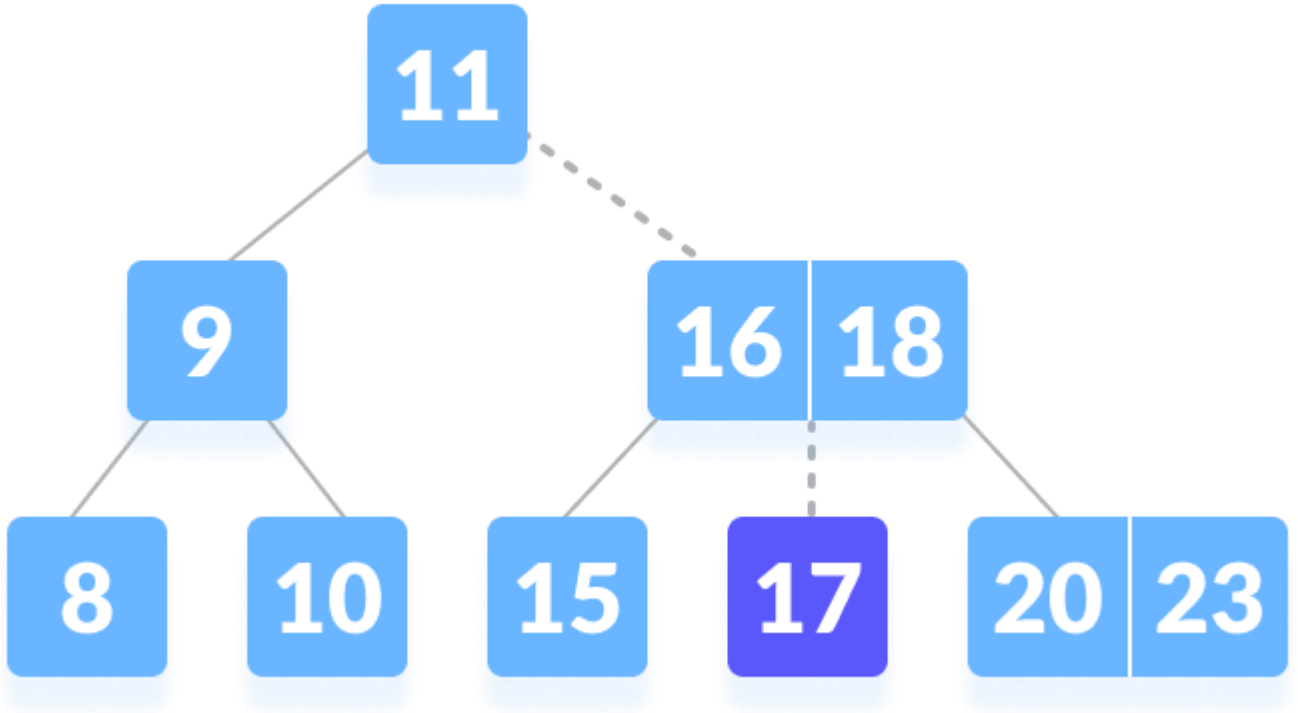
B Tree

Since k < 18, k lies between 16 and 18. Search in the right child of 16 or the left child of 18.



B Tree

k is found



B Tree - Insertion

Steps for Insertion -

- Locate the Leaf Node: Find the appropriate leaf node where the new key should be inserted.
- Insert in Sorted Order: Place the key in the correct position within the node.
- Check for Overflow: If the node exceeds the maximum allowed keys, it needs to be split.

Handling Overflow (Node Splitting)-

- Middle Key Promotion: If a node has more than 2t - 1 keys (where t is the minimum degree), split it into two.
- Split Process:
  - The median key moves up to the parent.
  - The left and right halves form two separate nodes.
- Recursion to Root: If the parent overflows, repeat the split upwards. If the root splits, the tree height increases.

B Tree - Insertion

- Edge Cases
  - Root Node Split: The only case where the height of the B-Tree increases.
  - Insertion in a Full Leaf Node: Requires splitting before inserting the key.
  - Multiple Splits: Can propagate up the tree if several nodes overflow.
- Complexity Analysis
  - Search for insertion: O(logn)
  - Insertion (with possible splits): O(logn)
  - Space Complexity: Efficient due to high branching factor.

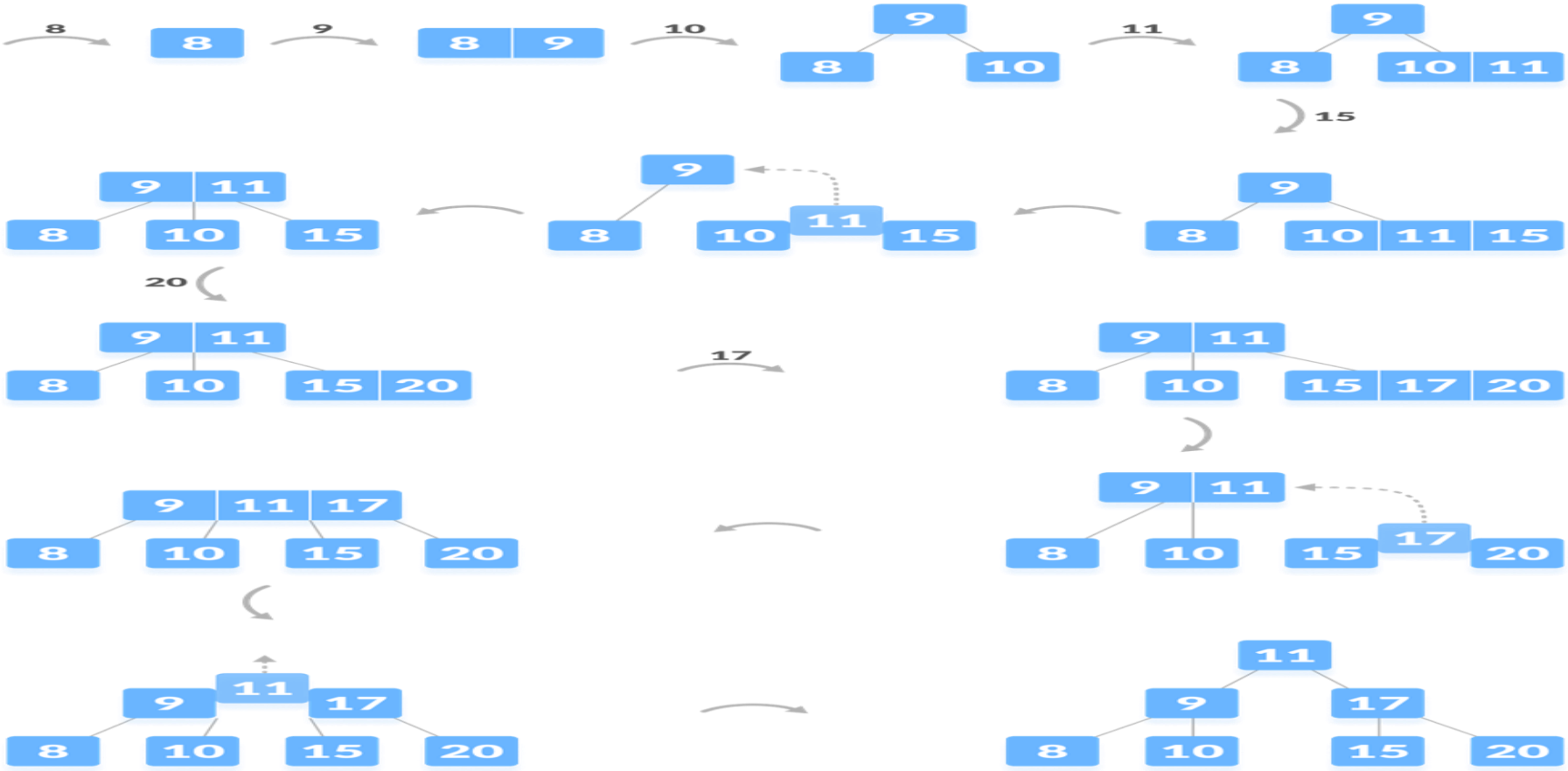
Minimum number of keys -> t-1, (m/2) - 1
Maximum number of keys -> 2t-1, m -1
Maximum number of children -> 2t, m
Where t is the minimum degree and m is the order of the tree

B Tree - Insertion Algorithm

```
1: procedure B-Tree-Insert (Node x, Key k)
2:    find i such that x:keys[i] > k or i >=numkeys(x)
3:    if x is a leaf then
4:       Insert k into x.keys at i
5:    else
6:      if x:child[i] is full then
7:         Split x:child[i]
8:         if k > x:key[i] then
9:            i = i + 1
10:        end if
11:      end if
12:     B-Tree-Insert(x:child[i]; k)
13:  end if
14: end procedure
```

DESIGN AND ANALYSIS OF ALGORITHMS

B Tree Insertion Example



Insertion Example

Let us understand the insertion operation with the illustrations below.
The elements to be inserted are 8, 9, 10, 11, 15, 16, 17, 18, 20, 23.

B Tree - Deletion

Deletion Operation on a B-Tree:

Deleting an element on a B-tree consists of three main events: **searching the node where the key to be deleted exists**, deleting the key and balancing the tree if required.

While deleting a node in a tree, a condition called **underflow** may occur. Underflow occurs when a node contains less than the minimum number of keys it should hold.

The terms to be understood before studying deletion operation are:

1. **Inorder Predecessor**

The largest key on the left child of a node is called its inorder predecessor.

2. **Inorder Successor**

The smallest key on the right child of a node is called its inorder successor.

B Tree - Deletion

Before going through the steps below, one must know these facts about a B tree of degree **m**.

1. A node can have a maximum of m children. (i.e. 3)
2. A node can contain a maximum of m - 1 keys. (i.e. 2)
3. A node should have a minimum of [m/2] children. (i.e. 2)
4. A node (except root node) should contain a minimum of [m/2] - 1 keys. (i.e. 1)

There are three main cases for deletion operation in a B tree.

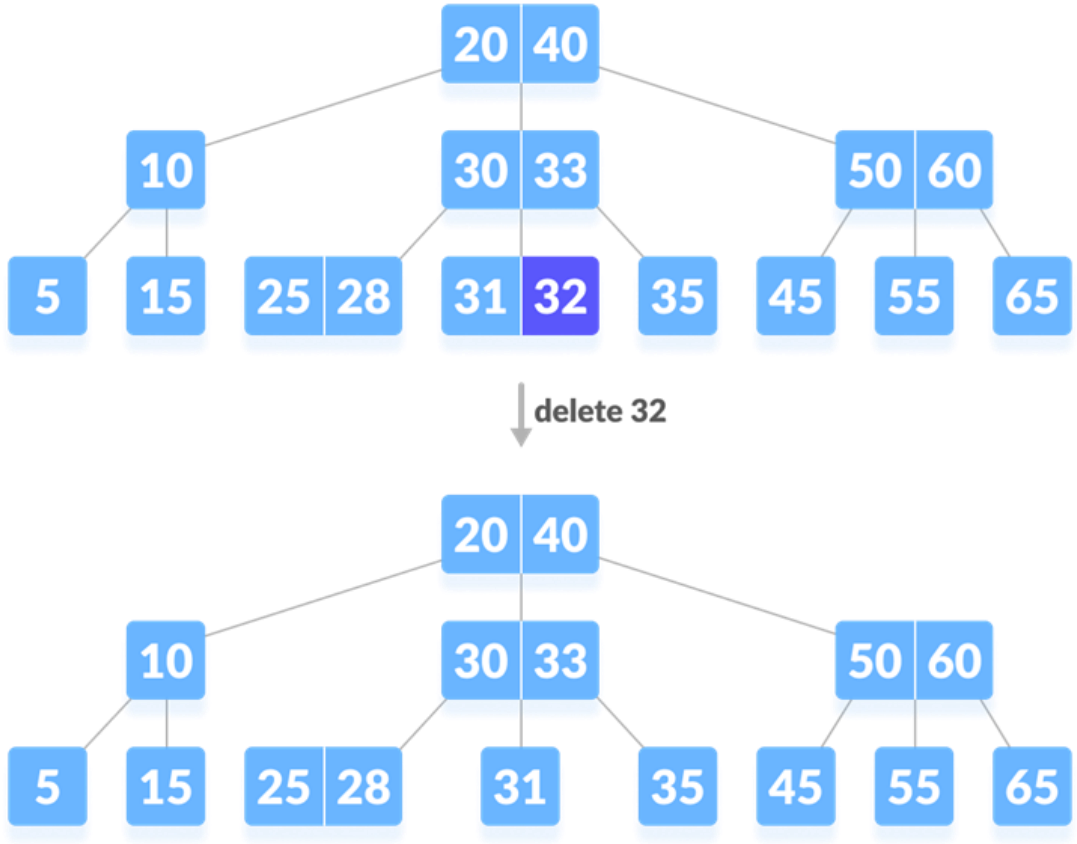
B Tree - Deletion

Case I

The key to be deleted lies in the leaf. There are two cases for it.

1.The deletion of the key does not violate the property of the minimum number of keys a node should hold.

In the tree below, deleting 32 does not violate the above properties.



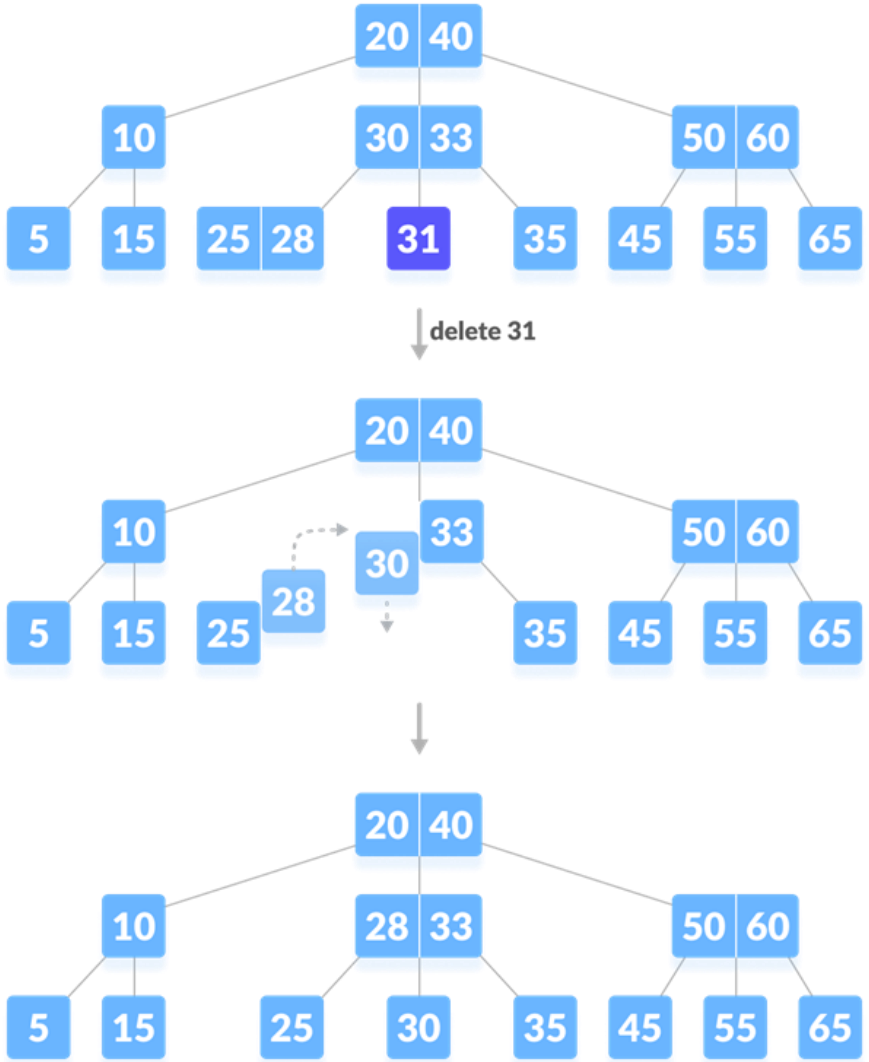
B Tree - Deletion

2. The deletion of the key violates the property of the minimum number of keys a node should hold. In this case, we borrow a key from its immediate neighboring sibling node in the order of left to right.

First, visit the immediate left sibling. If the left sibling node has more than a minimum number of keys, then borrow a key from this node.

Else, check to borrow from the immediate right sibling node.

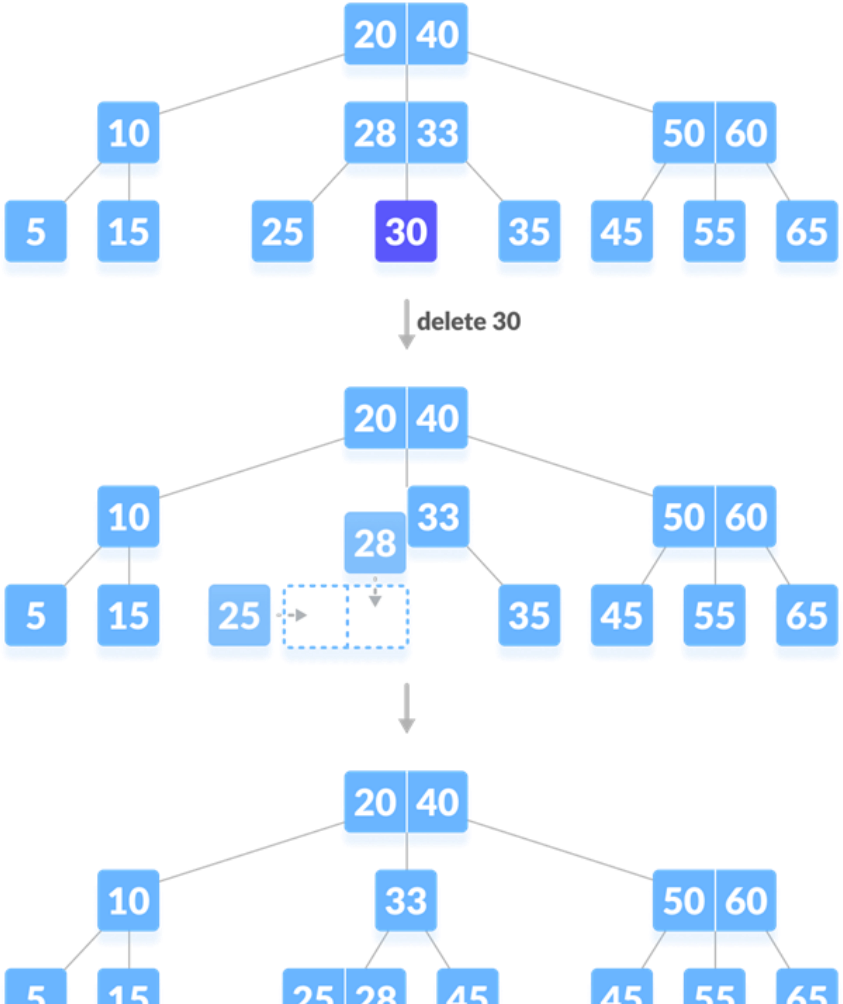
In the tree below, deleting 31 results in the above condition. Let us borrow a key from the left sibling node.



B Tree - Deletion

If both the immediate sibling nodes already have a minimum number of keys, then merge the node with either the left sibling node or the right sibling node. **This merging is done through the parent node.**

Deleting 30 results in the above case.

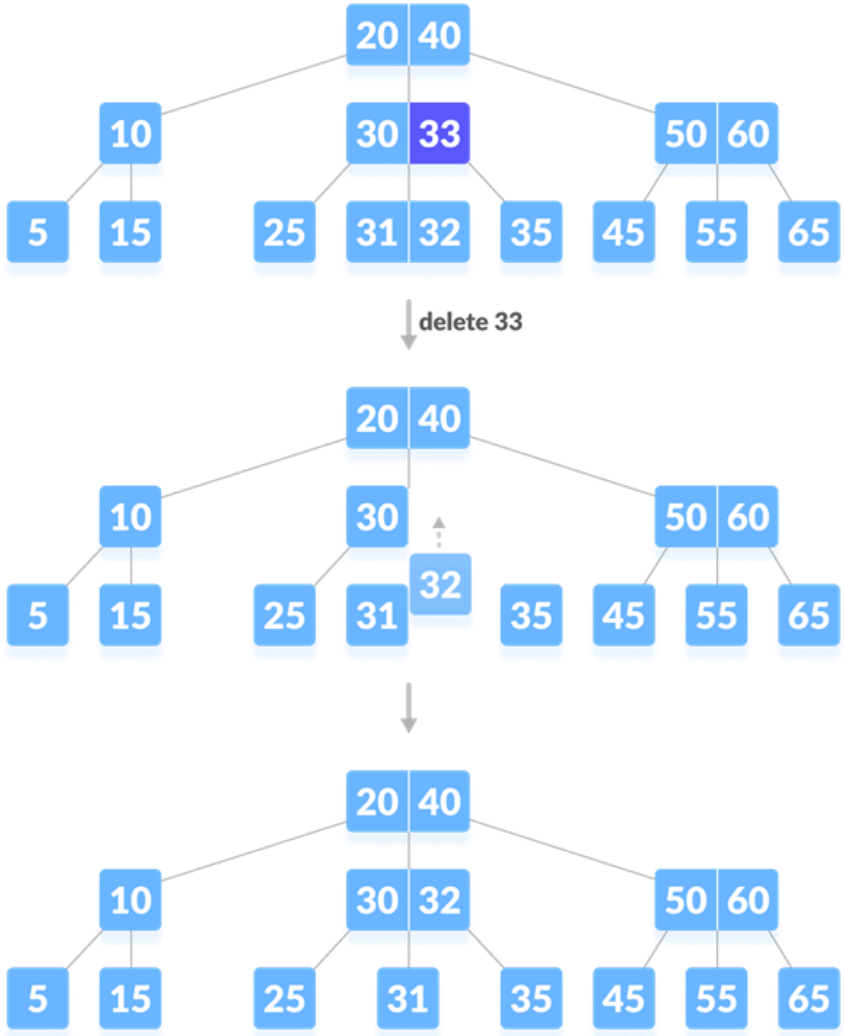


B Tree - Deletion

Case II

If the key to be deleted lies in the internal node, the following cases occur.

1.The internal node, which is deleted, is replaced by an inorder predecessor if the left child has more than the minimum number of keys.

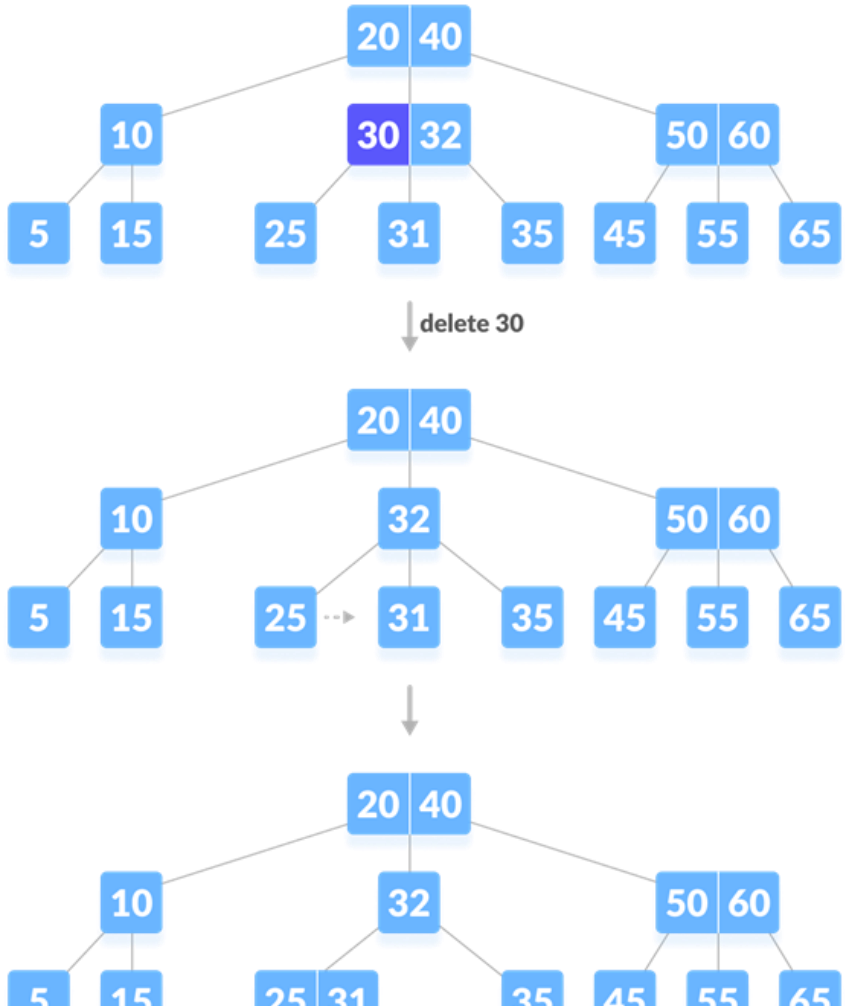


B Tree - Deletion

2. The internal node, which is deleted, is replaced by an inorder successor if the right child has more than the minimum number of keys.

3. If either child has exactly a minimum number of keys then, merge the left and the right children.

After merging if the parent node has less than the minimum number of keys then, look for the siblings as in Case I.

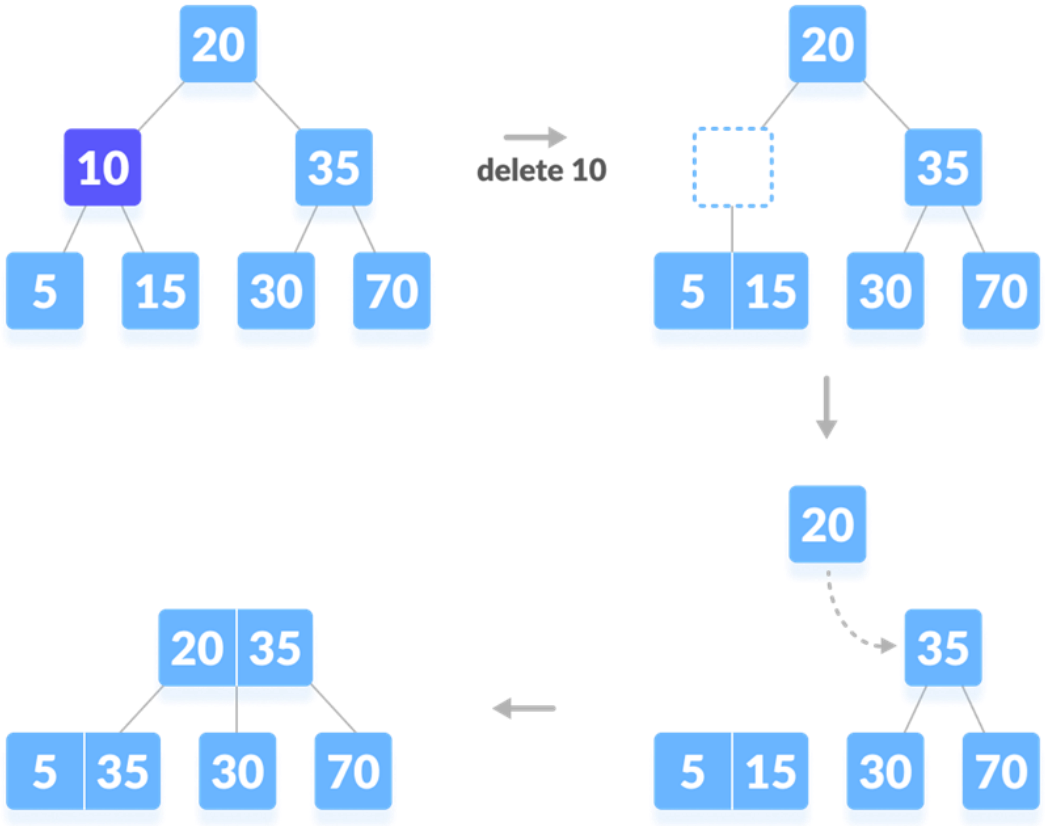


B Tree - Deletion

Case III

In this case, the height of the tree shrinks. If the target key lies in an internal node, and the deletion of the key leads to a fewer number of keys in the node (i.e. less than the minimum required), then look for the inorder predecessor and the inorder successor. If both the children contain a minimum number of keys then, borrowing cannot take place. This leads to Case II(3) i.e. merging the children.

Again, look for the sibling to borrow a key. But, if the sibling also has only a minimum number of keys then, merge the node with the sibling along with the parent. Arrange the children accordingly (increasing order).



B Tree - Deletion

Deletion Complexity

Best case Time complexity: Θ(log n)

Average case Space complexity: Θ(n)

Worst case Space complexity: Θ(n)

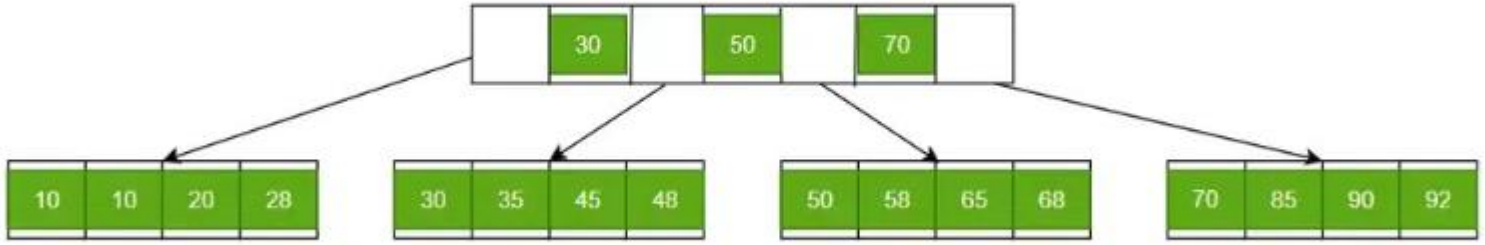
PES UNIVERSITY
CELEBRATING 50 YEARS

DESIGN AND ANALYSIS OF ALGORITHMS

B+ Trees
(additional Information- not in syllabus)

B+ Tree

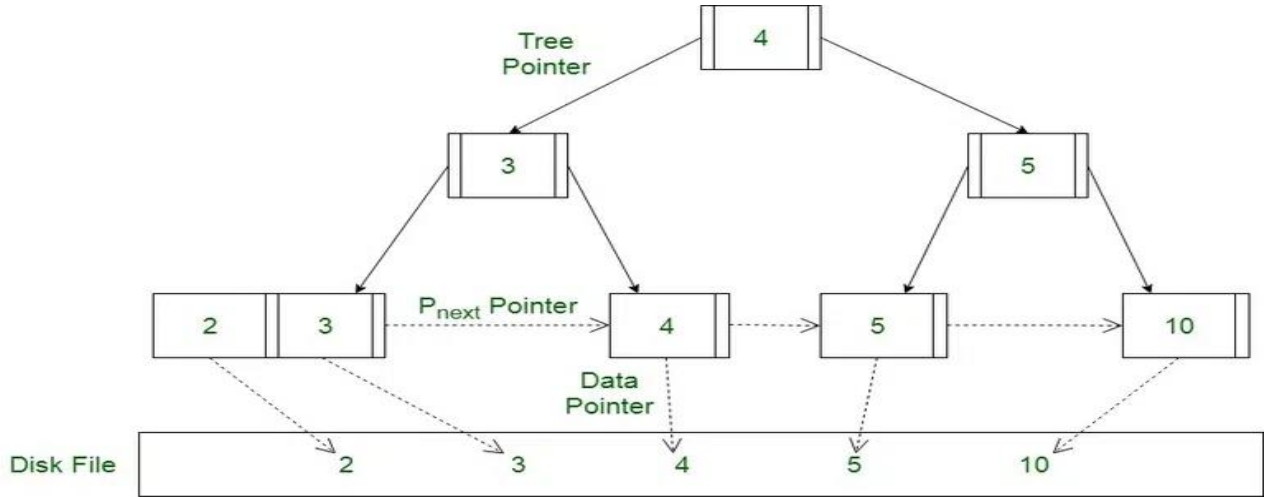
- B + Tree is a variation of the B-tree data structure. In a B + tree, data pointers are stored only at the leaf nodes of the tree.
- In a B+ tree structure of a leaf node differs from the structure of internal nodes.
  - The leaf nodes have an entry for every value of the search field, along with a data pointer to the record (or to the block that contains this record). The leaf nodes of the B+ tree are linked together to provide ordered access to the search field to the records.
  - Internal nodes of a B+ tree are used to guide the search. Some search field values from the leaf nodes are repeated in the internal nodes of the B+ tree.



B+ Tree

- Properties of B+ Trees
  - Each node except root can have a maximum of M children and at least ceil(M/2) children.
  - Each node can contain a maximum of M – 1 keys and a minimum of ceil(M/2) – 1 keys.
  - The root has at least two children and atleast one search key.
  - While insertion overflow of the node occurs when it contains more than M – 1 search key values

Here M is the order of the B Tree



B+ Tree

Operations on a B+ Tree:

- Insertion
- Deletion

B+ Tree - Insertion

Properties for insertion B+ Tree

- Case 1: Overflow in leaf node
  - Split the leaf node into two nodes.
  - First node contains ceil((m-1)/2) values.
  - Second node contains the remaining values.
  - Copy the smallest search key value from second node to the parent node.(Right biased)

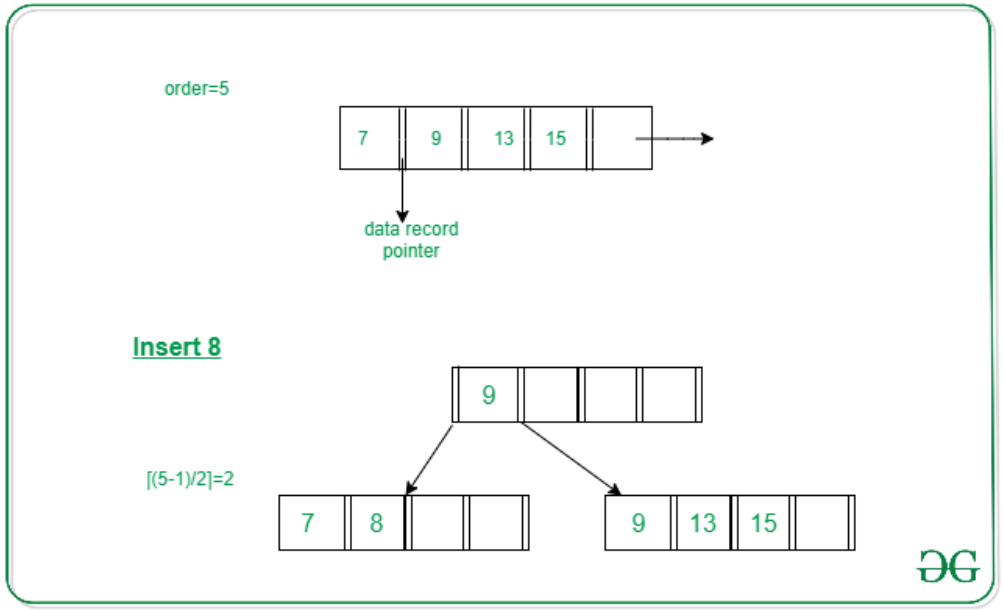


Illustration of inserting 8 into B+ Tree of order of 5

B+ Tree - Insertion

Properties for insertion B+ Tree

- Case 2: Overflow in non-leaf node
  - Split the non leaf node into two nodes.
  - First node contains ceil(m/2)-1 values.
  - Move the smallest among remaining to the parent.
  - Second node contains the remaining keys.

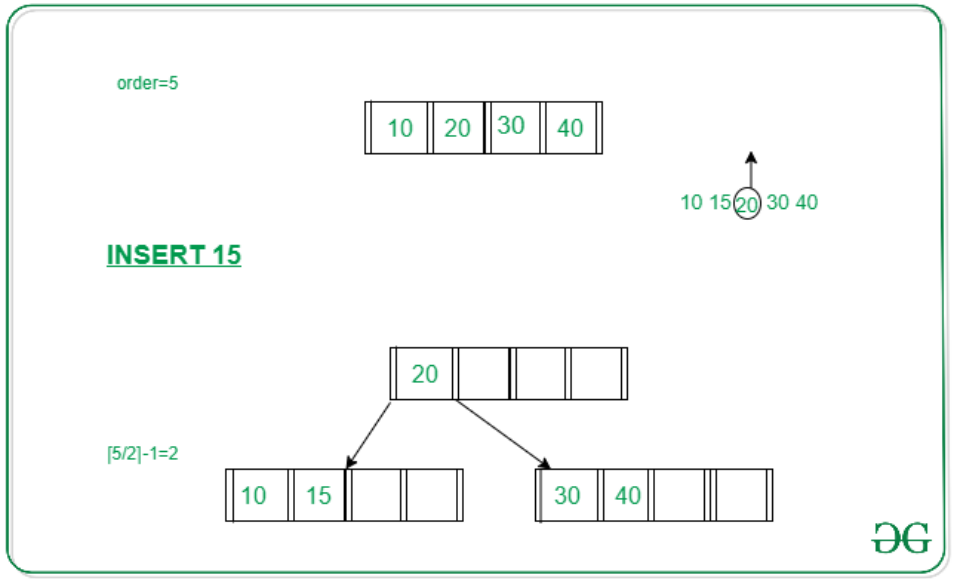
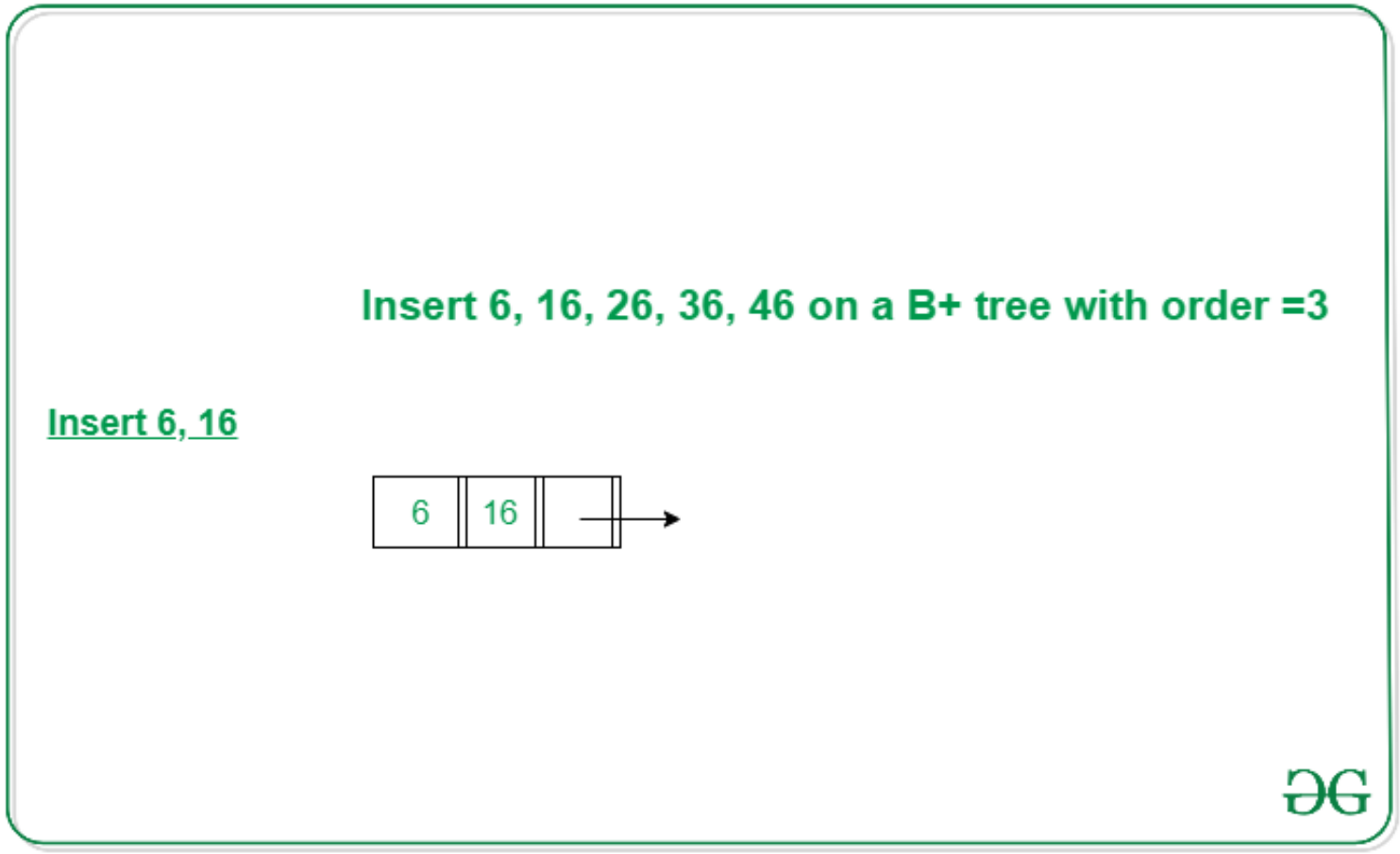


Illustration of inserting 15 into B+ Tree of order of 5

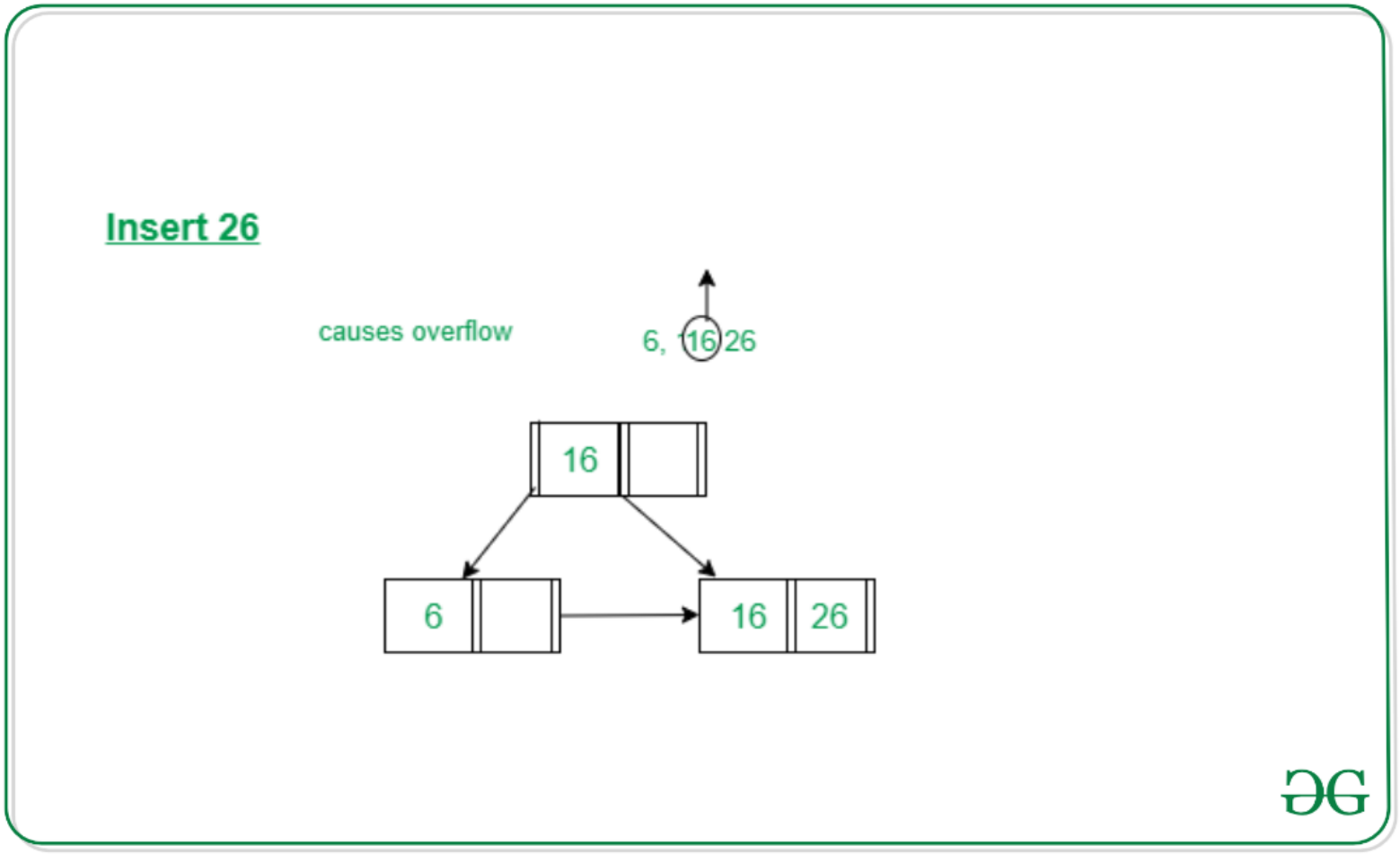
B+ Tree - Insertion Example

Problem: Insert the following key values 6, 16, 26, 36, 46 on a B+ tree with order = 3.



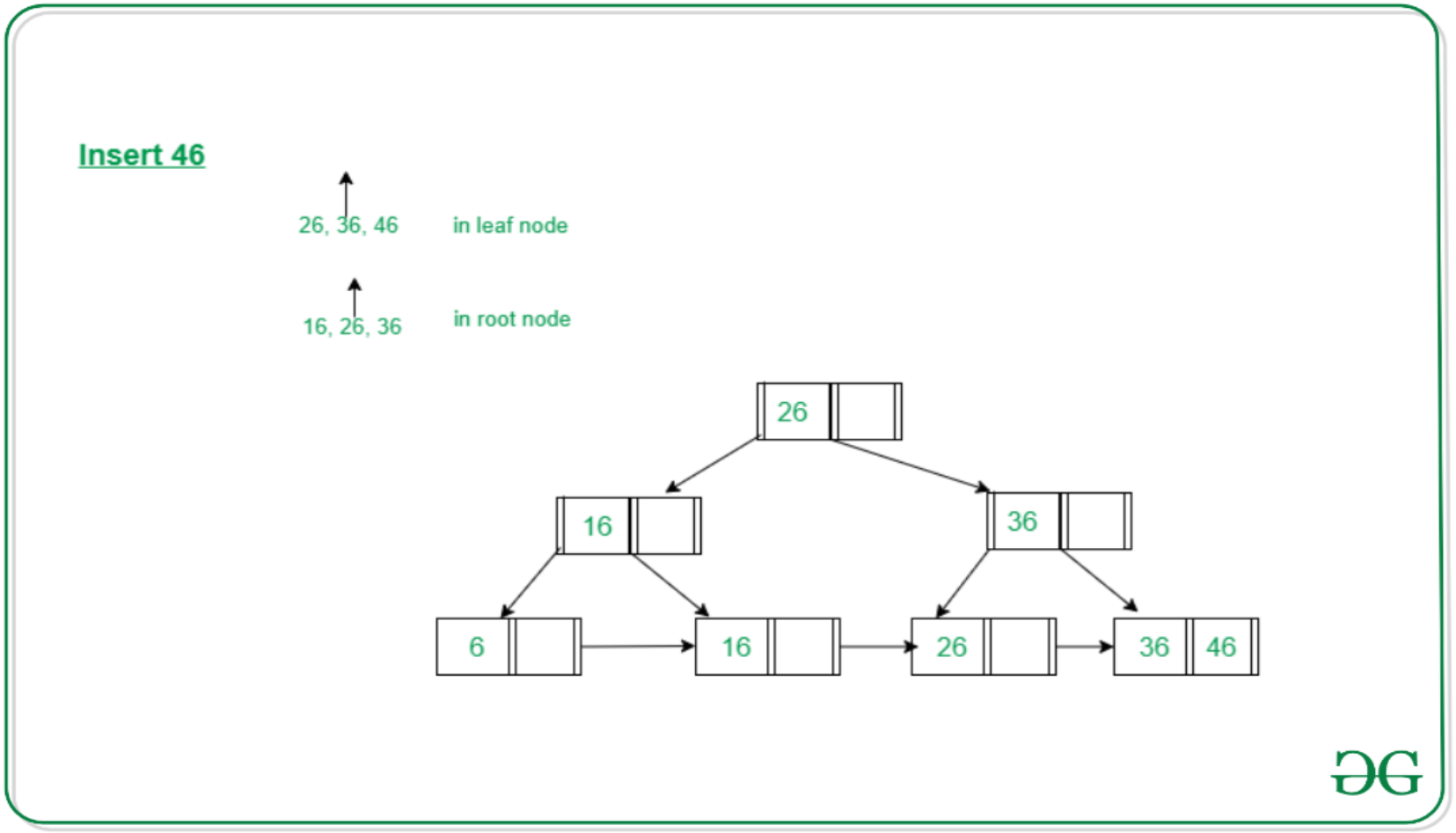
B+ Tree - Insertion Example

Problem: Insert the following key values 6, 16, 26, 36, 46 on a B+ tree with order = 3.



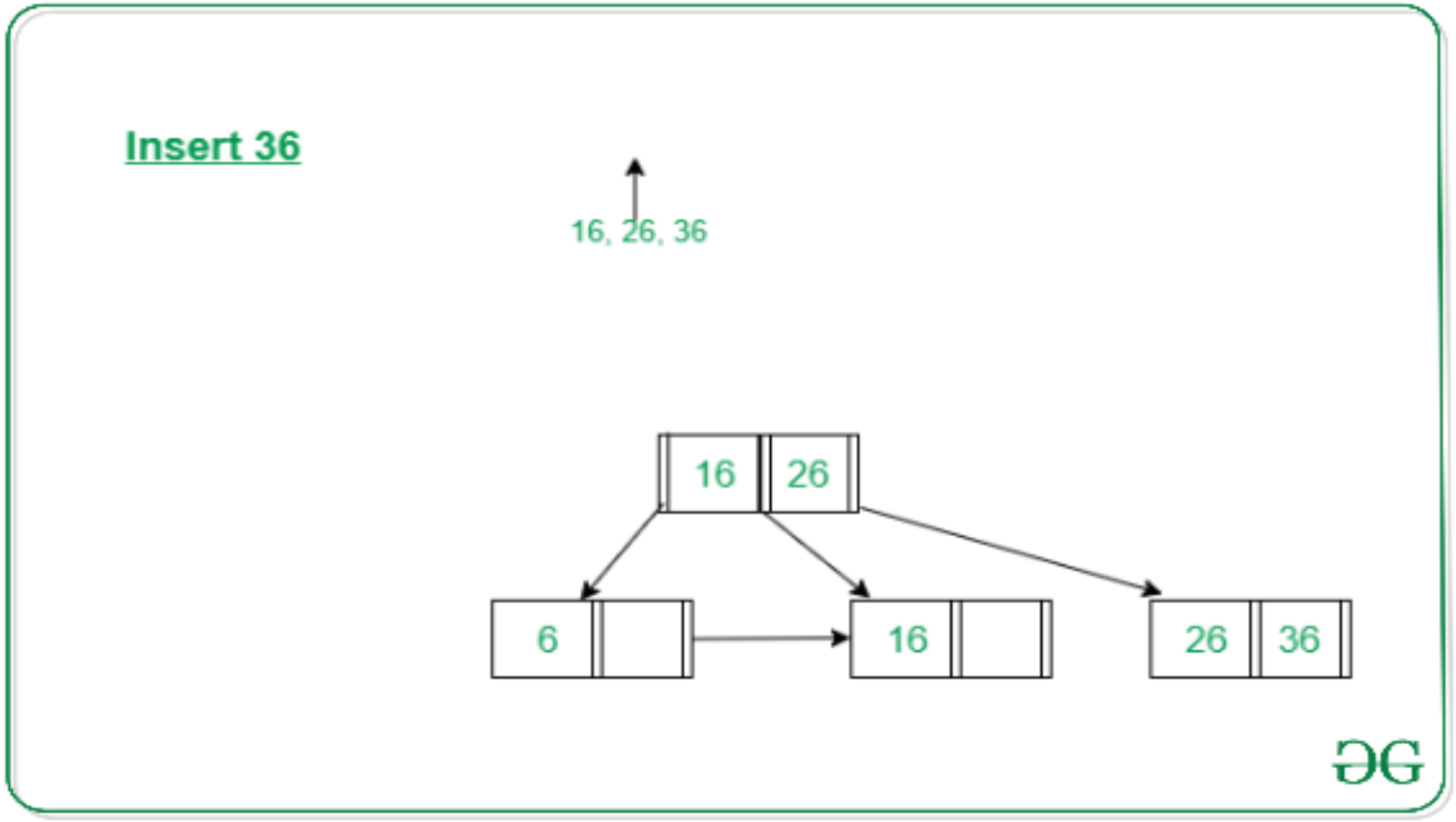
B+ Tree - Insertion Example

Problem: Insert the following key values 6, 16, 26, 36, 46 on a B+ tree with order = 3.



B+ Tree - Insertion Example

Problem: Insert the following key values 6, 16, 26, 36, 46 on a B+ tree with order = 3.



B+ Tree - Insertion Example

Problem: Insert the following key values 5,15, 25, 35, 45 on a B+ tree with order = 3.

1) Insert 5

B+ Tree - Insertion Example

Problem: Insert the following key values 5,15, 25, 35, 45 on a B+ tree with order = 3.

2) Insert 15

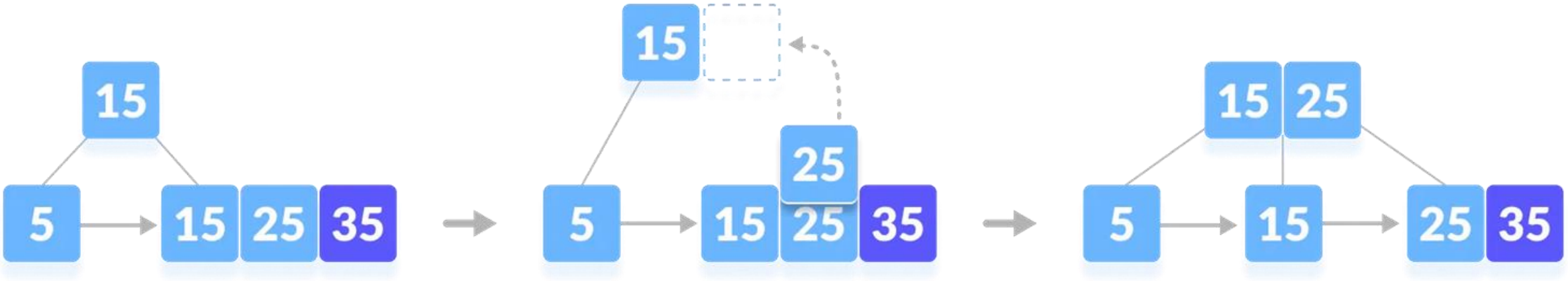
B+ Tree - Insertion Example

Problem: Insert the following key values 5,15, 25, 35, 45 on a B+ tree with order = 3.

3) Insert 25

B+ Tree - Insertion Example

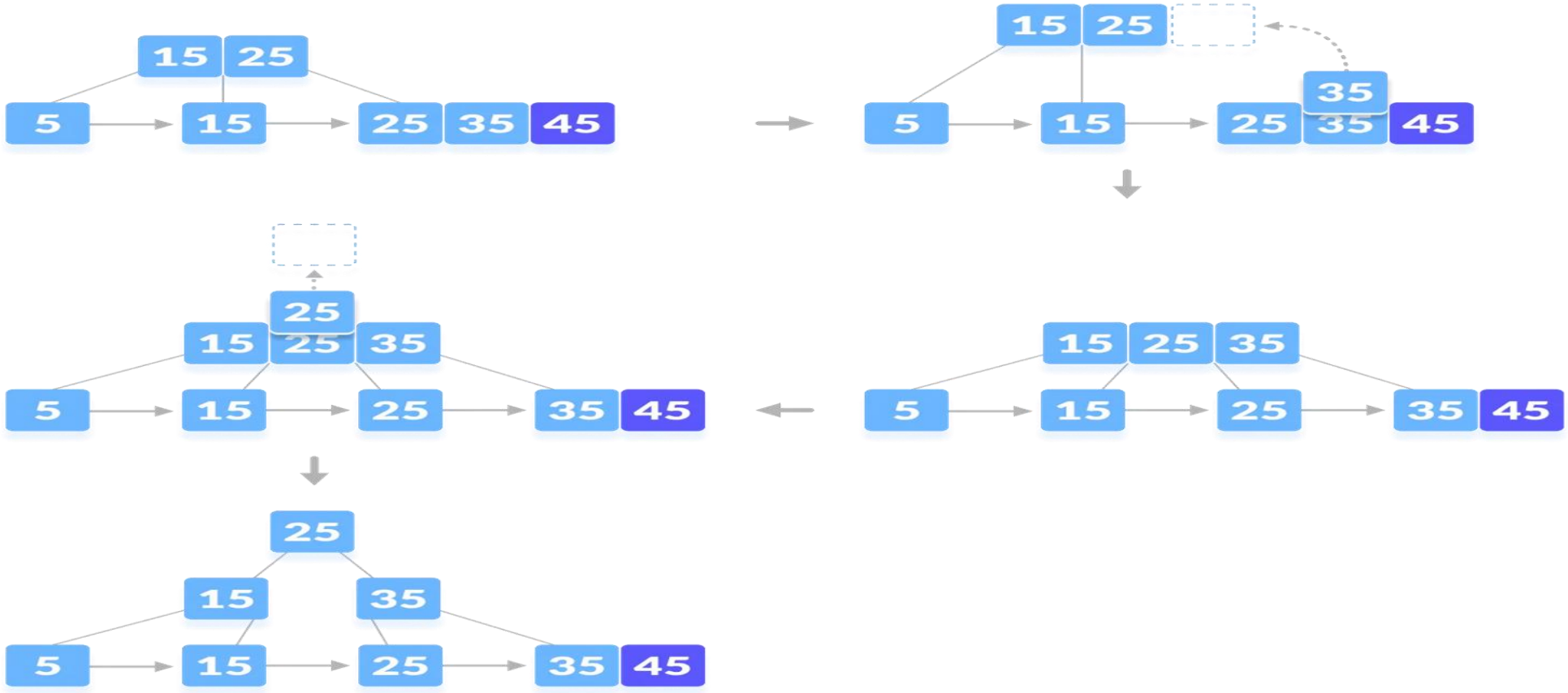
Problem: Insert the following key values 5,15, 25, 35, 45 on a B+ tree with order = 3.



4) Insert 35

B+ Tree - Insertion Example

Problem: Insert the following key values 5,15, 25, 35, 45 on a B+ tree with order = 3.



4) Insert 45

B+ Tree - Deletion

- Deleting an element on a B+ tree consists of three main events: searching the node where the key to be deleted exists, deleting the key and balancing the tree if required.
- Underflow is a situation when there is less number of keys in a node than the minimum number of keys it should hold.
- While deleting a key, we have to take care of the keys present in the internal nodes (i.e. indexes) as well because the values are redundant in a B+ tree.
  - Search the key to be deleted then follow the following steps.

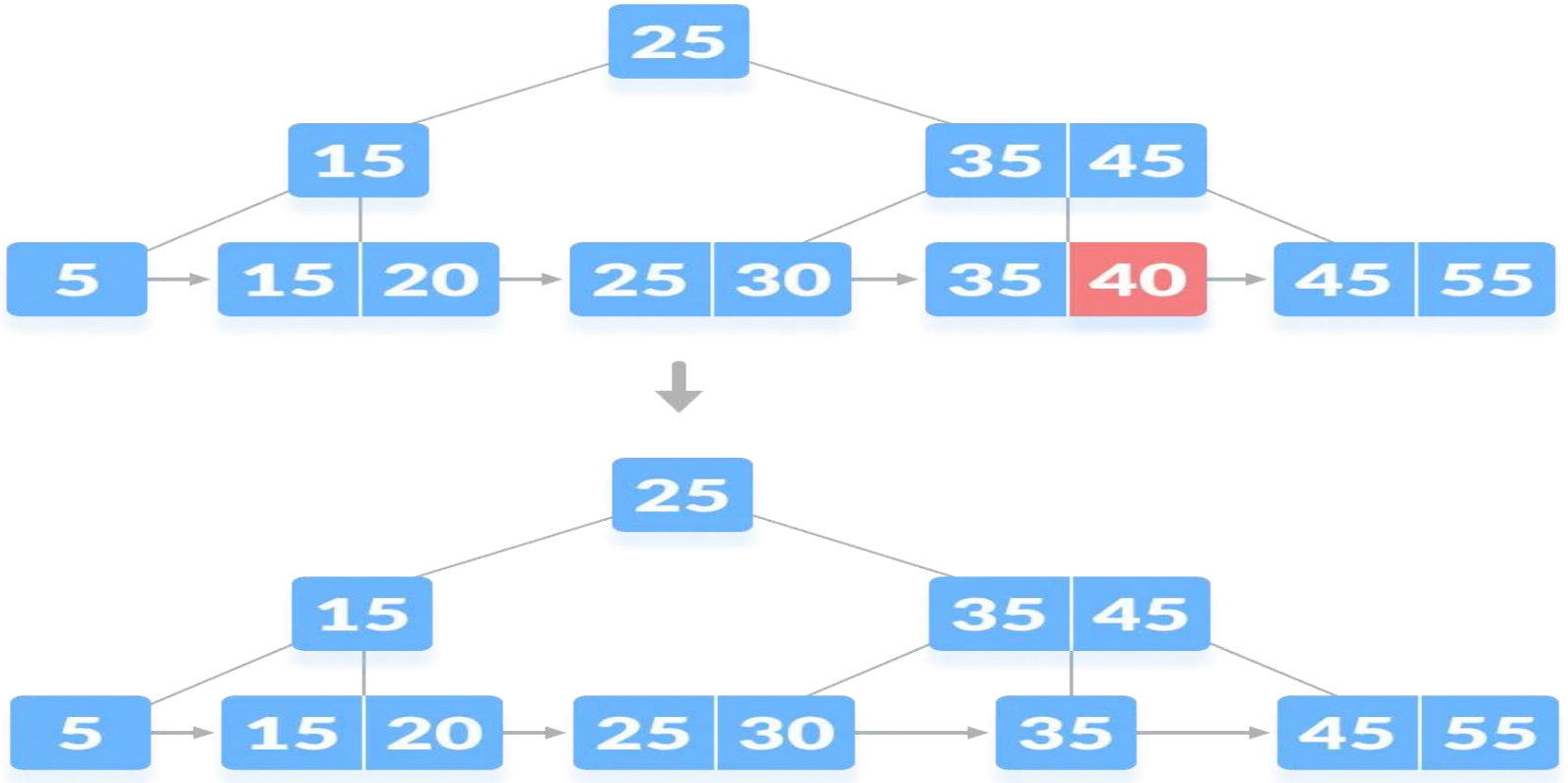
B+ Tree - Deletion Strategy

- Look to locate the deleted key in the leaf nodes.
- Delete the key and its associated value if the key is discovered in a leaf node.
- One of the following steps should be taken if the node underflows (number of keys is less than half the maximum allowed):
  - Get a key by borrowing it from a sibling node if it contains more keys than the required minimum.
  - If the minimal number of keys is met by all of the sibling nodes, merge the underflow node with one of its siblings and modify the parent node as necessary.
- Remove all references to the deleted leaf node from the internal nodes of the tree.
- Remove the old root node and update the new one if the root node is empty.

B+ Tree - Deletion Case 1

- Case 1 - The key to be deleted is present only at the leaf node not in the indexes (or internal nodes). There are two cases -
  - Case 1.1 - There is more than the minimum number of keys in the node. Simply delete the key

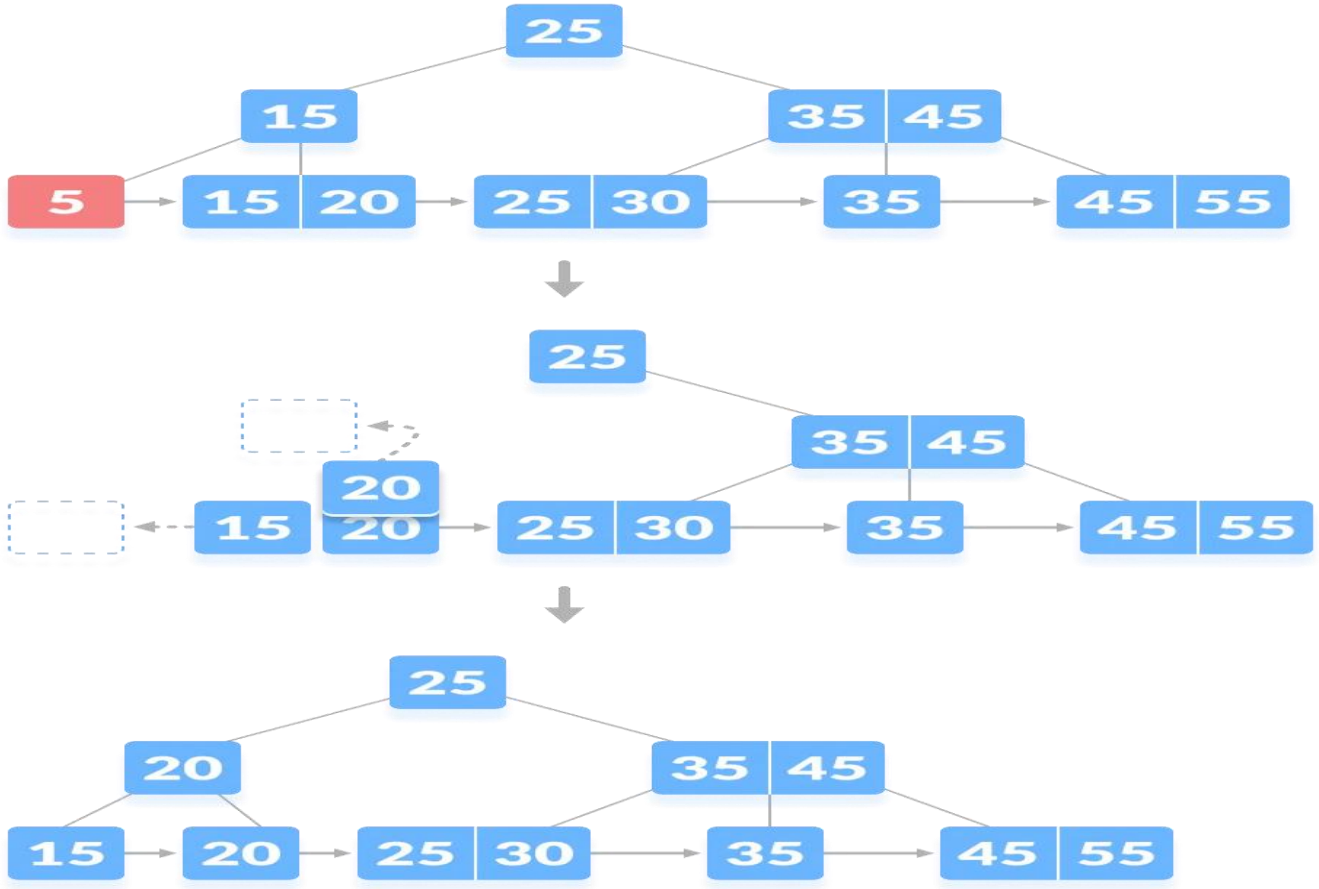
Deleting 40



B+ Tree - Deletion Case 1

- Case 1.2 - There is an exact minimum number of keys in the node. Delete the key and borrow a key from the immediate sibling. Add the median key of the sibling node to the parent.

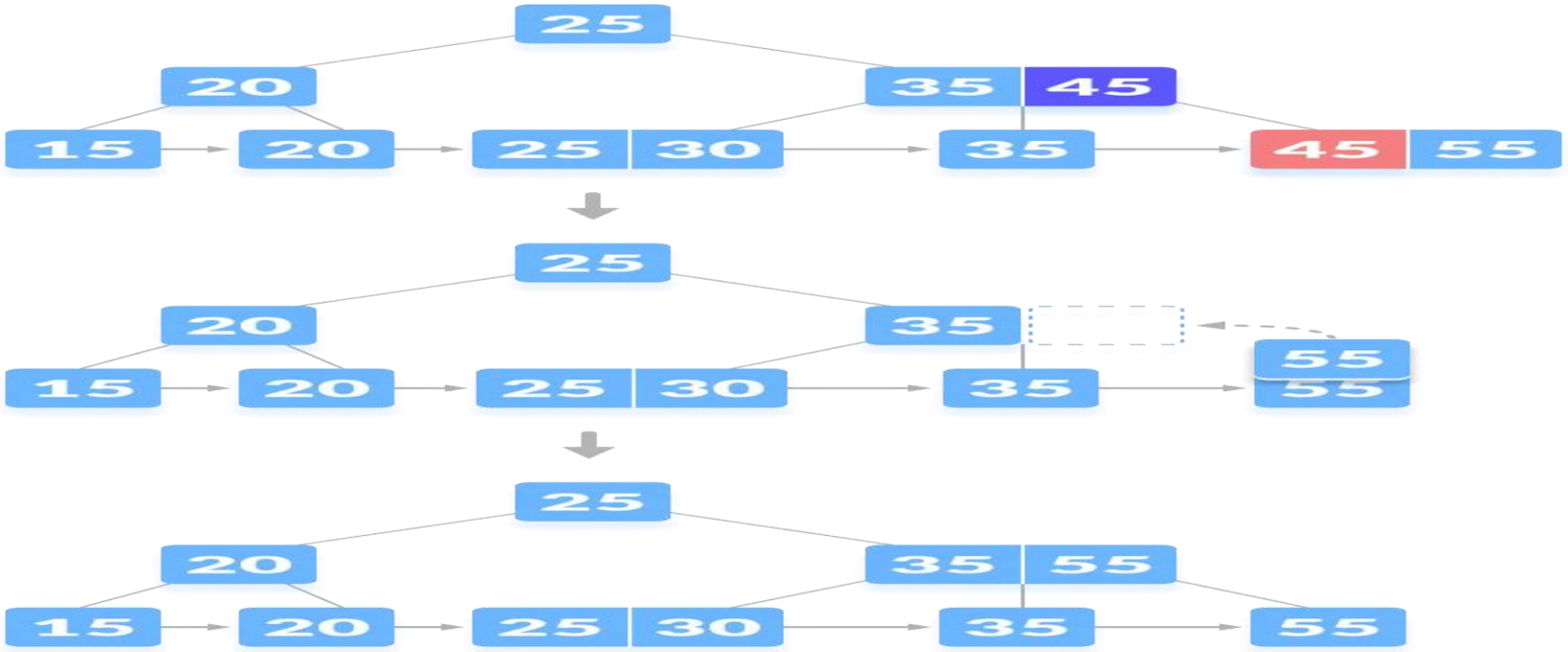
Deleting 5



B+ Tree - Deletion Case 2

- Case 2 - The key to be deleted is present in the internal nodes as well. Then we have to remove them from the internal nodes as well. There are three cases -
  - Case 2.1 - If there is more than the minimum number of keys in the node, simply delete the key from the leaf node and delete the key from the internal node as well. Fill the empty space in the internal node with the inorder successor.

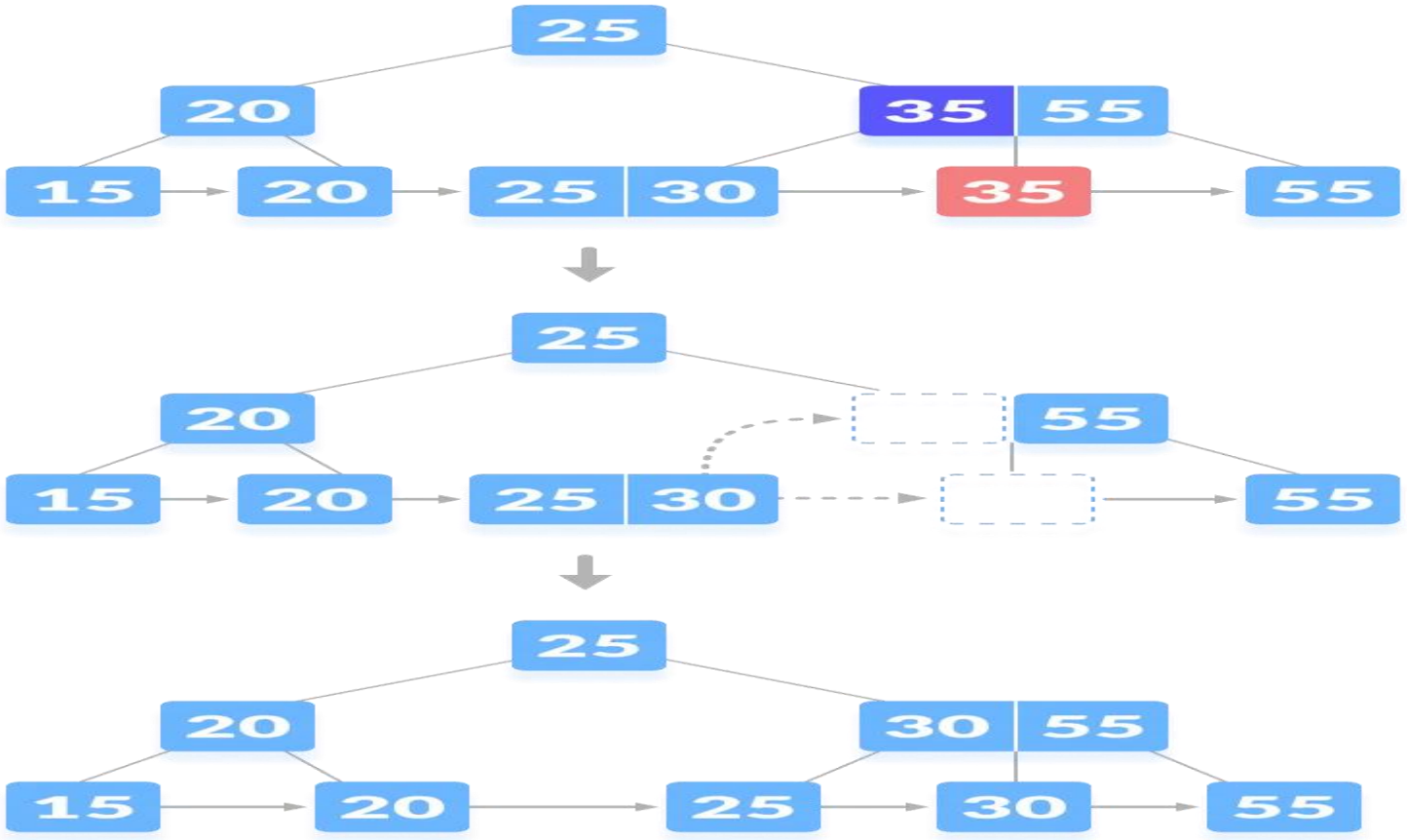
Deleting 45



B+ Tree - Deletion Case 2

- Case 2.2 - If there is an exact minimum number of keys in the node, then delete the key and borrow a key from its immediate sibling (through the parent). Fill the empty space created in the index (internal node) with the borrowed key.

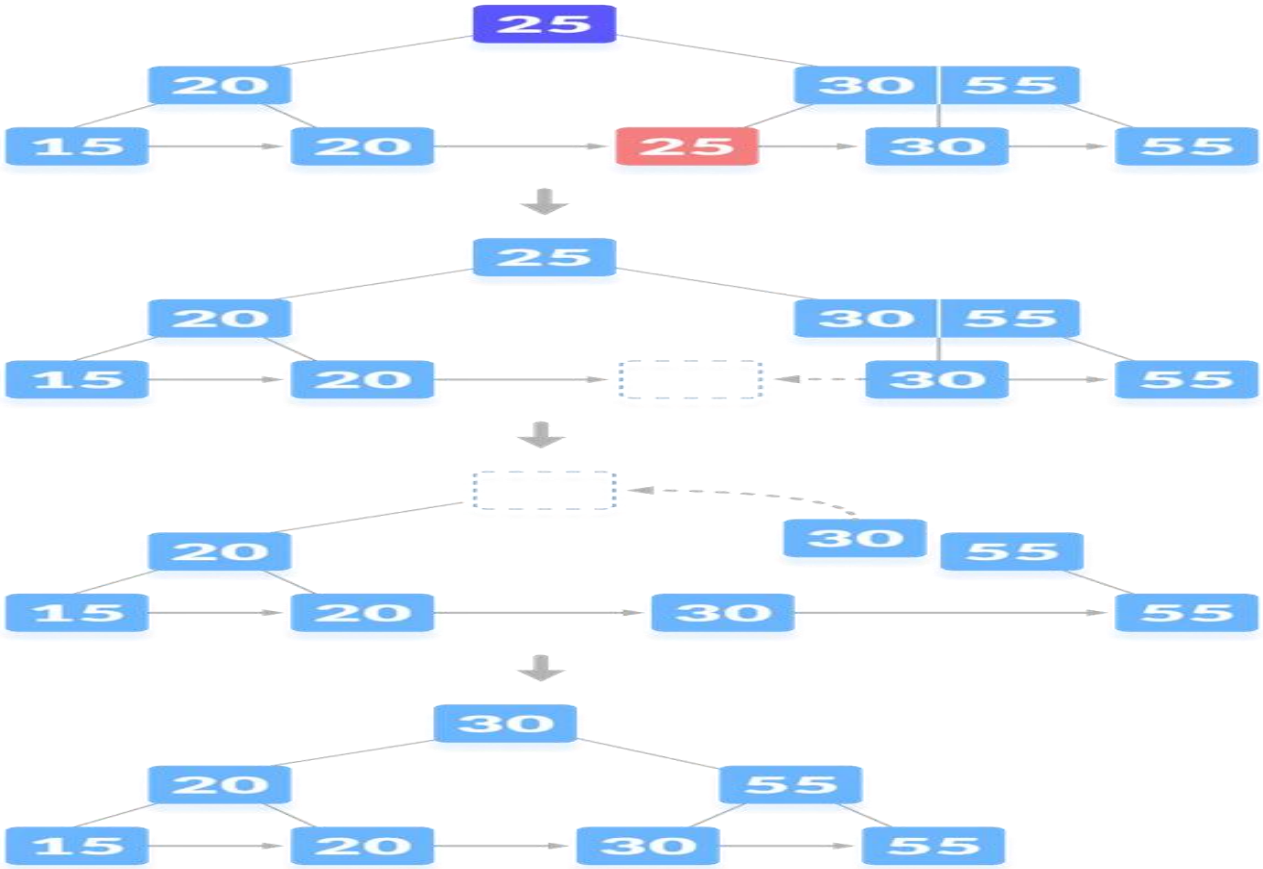
Deleting 35



B+ Tree - Deletion Case 2

- Case 2.3 - This case is similar to Case II(1) but here, empty space is generated above the immediate parent node. After deleting the key, merge the empty space with its sibling. Fill the empty space in the grandparent node with the inorder successor.

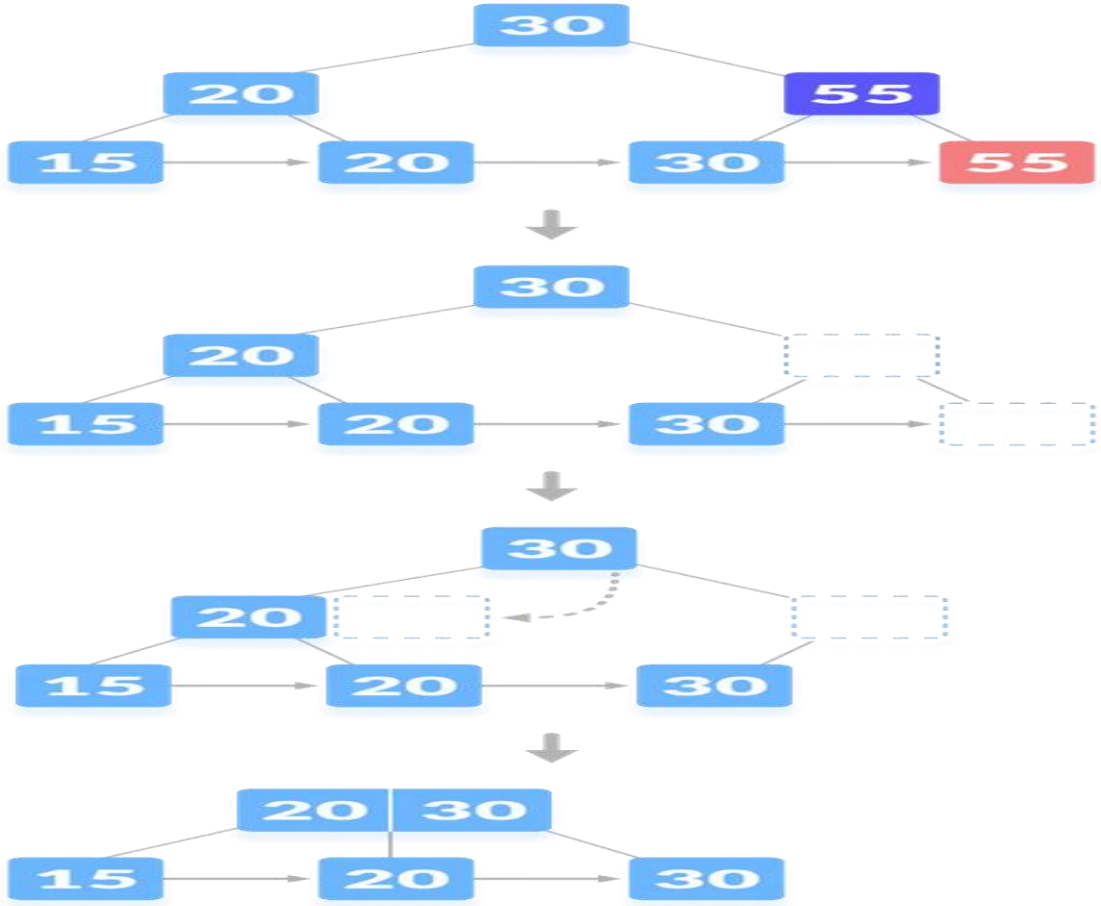
Deleting 25



B+ Tree - Deletion Case 3

- Case 3 - In this case, the height of the tree gets shrinked. It is a little complicated.Deleting 55 from the tree below leads to this condition.

Deleting 55



B+ Tree - Insertion and Deletion Complexity

- Insertion
  - Time complexity: O(log n)
  - Auxiliary Space: O(log n)

- Deletion
  - Time Complexity: O(log N)
  - Auxiliary Space: O(N)

DESIGN AND ANALYSIS OF ALGORITHMS

Difference between B Tree and B+ Tree

B Tree	B+ Tree
Nodes store both keys and data values	Separate leaf nodes for data storage and internal nodes for indexing
Leaf nodes do not form a linked list	Leaf nodes form a linked list for efficient range-based queries
Lower order (fewer keys)	Higher order (more keys)
Usually does not allow key duplication	Typically allows key duplication in leaf nodes
Deletion of the internal node is very complex and the tree has to undergo a lot of transformations.	Deletion of any node is easy because all node are found at leaf.
Insertion takes more time and it is not predictable sometimes.	Insertion is easier and the results are always the same.
Requires less memory as keys and values are stored in the same node	Requires more memory for internal nodes
Use Cases - In-memory data structures, databases, general-purpose use	Use cases - Database systems, file systems, where range queries are common

PES UNIVERSITY
CELEBRATING 50 YEARS

References

https://www.programiz.com/dsa/deletion-from-a-b-tree
https://www.programiz.com/dsa/insertion-into-a-b-tree
https://www.geeksforgeeks.org/insert-operation-in-b-tree/
https://www.geeksforgeeks.org/introduction-of-b-tree/
https://www.geeksforgeeks.org/difference-between-b-tree-and-b-tree/?ref=lbp
https://www.geeksforgeeks.org/insertion-in-a-b-tree/
https://www.programiz.com/dsa/insertion-on-a-b-plus-tree
https://www.programiz.com/dsa/deletion-from-a-b-plus-tree
https://www.geeksforgeeks.org/deletion-in-b-tree/

THANK YOU

Teaching Assistants

Suma Grandhi

Vaishnavi M